SimPhysQA: Evidence-Bound Language Annotation of Physical Simulations for Grounded Scientific Question Answering

Anonymous ACL submission

Abstract

Simulator-grounded question answering is not new: physical-reasoning benchmarks already pair simulated videos and driving scenes with generated questions. What remains scarce is a reusable language layer, that describes dynamical rollout at trajectory and timestamp level, with descriptions. We introduce SIMPHYSQA, a resource and annotation pipeline that adds this layer to physics-simulator trajectories from four domains: robotic arm, driving, object pushing/stacking, and tethered loads. An LLM writes trajectory summaries and per-timestamp descriptions from a projection that contains only permitted public evidence. Each sentence carries JSON-pointer bindings to source fields, deterministic code adds event and controller descriptions, and validators check bindings, numerical fidelity, units, frames, timestamps, and specified unsupported-assertion patterns. In a focused audit, three reviewers independently judged 100%, 83%, and 87% of 140 annotation packets acceptable. On these records, SIMPHYSQA builds five question families, each with its own public/private evidence contract: direct observation, prediction, counterfactual intervention backed by matched re-simulation, explanation of a supplied comparison, and clarification of under-specified queries. We evaluate nine local, tool-free LLM configurations of five models, plus two matched language ablations, on a balanced 200-item core. End-to-end success reaches 92.5% on direct readout but at most 12.5% on prediction and 35.0% on counterfactuals. These failures split into abstentions, output-contract violations, and incorrect values. Removing only the generated language lowers Qwen3-8B's direct-readout success by 20.0 points. We document every executed prompt, schema, and setting so that the annotation and the evaluation can be reproduced.

1 Introduction

A scientist, roboticist, or simulation engineer often asks questions whose answer is neither a memorized fact nor a label visible in one frame. Where will the payload be one second after the observed cutoff? Would the block still fall if the push were omitted? Did it succeed? Answering requires connecting a query to a time-indexed record of states, controls, and scheduled events, respecting the simulation's units and control assumptions, and either answering with evidence or asking for the missing information.

Simulator-grounded QA has clear precedent (§2, Table 1). In existing resources, however, language is produced to pose and answer individual questions. What is missing is a reusable language layer for the trajectory itself: descriptions at the trajectory and timestamp level, each tied to specific simulator fields, together with explicit rules about which evidence each type of question may reveal. Building this layer is not captioning. A description of a realized future state leaks a prediction answer, whereas a control scheduled for the future is legitimately public. A scheduled push is not an observed push, and timing alone does not establish cause. Fluent but weakly grounded language lets a model sound right while using the wrong time, entity, unit, or frame.

We introduce SIMPHYSQA, a resource built on an evidence-bound annotation pipeline for physics simulations in four domains (§3; overview in Figure 1, Appendix A). An LLM writes trajectory and per-timestamp descriptions from public evidence only, and each sentence is bound to the fields it uses. Deterministic code then adds event descriptions with exact temporal status, and validators check what the text asserts. Because these checks do not prove entailment, a focused three-reviewer audit assesses residual errors; reviewers judged 83–100% of packets acceptable. On these records, SIMPHYSQA builds five question families (direct, predictive, counterfactual, explanation, and clarification), each with its own public/private evidence contract (§4).

Evaluating nine local, tool-free configurations of five LLMs shows why the families must stay separate (§6). The best configuration reads observed state with 92.5% success, yet none exceeds 12.5% on prediction, and these failures split into abstentions, contract violations, and incorrect values. Removing only the generated language lowers Qwen3-8B's direct-readout success by 20.0 points.

Contributions. (1) **An annotation pipeline** that produces trajectory-, timestamp-, and event-level descriptions with explicit simulator-field bindings. It separates control of the generator's evidence from automatic support checks and documents every executed prompt. (2) **A five-family QA resource** over four physical domains with per-family evidence contracts. (3) **A fixed-denominator evaluation** of nine configurations and two matched language ablations that separates abstentions, contract failures, and incorrect values.

2 Related Work

Physical reasoning in language. PIQA and NEWTON test physical commonsense and object attributes in text (Bisk et al., 2020; Wang et al., 2023b). ScienceQA and SciBench test exam-style and college-level science problems with lectures or solutions (Lu et al., 2022; Wang et al., 2023a). CLadder poses formal causal inference in language (Jin et al., 2023). None of these exposes a time-indexed physical state against which an answer can be checked.

Simulator-grounded QA. PHYRE and Physion evaluate physical prediction from rollouts and video (Bakhtin et al., 2019; Bear et al., 2021). CLEVRER, ComPhy, and CRAFT generate descriptive, predictive, and counterfactual questions over simulated videos (Yi et al., 2020; Chen et al., 2022; Ates et al., 2022), and PhysBench extends this line to vision-language models (Chow et al., 2025). Bench2Drive-VL shares our motivation of generating language from simulator state: it derives behavior-grounded, per-frame questions from privileged CARLA state for closed-loop evaluation of driving models (Jia et al., 2026). SIMPHYSQA instead produces reusable trajectory- and timestamp-level descriptions with explicit field bindings across four physical domains, and it specifies for each question family which evidence is public and which is private. Mind's Eye uses a simulator as a tool at inference time (Liu et al., 2022); SIMPHYSQA evaluates tool-free answering.

Ambiguity and faithful evaluation. AmbigQA, ShARC, and work on clarifying questions study under-specified queries in open-domain text and rule-based settings (Min et al., 2020; Saeidi et al., 2018; Aliannejadi et al., 2019); SIMPHYSQA brings clarification to physical QA. Our reporting follows recommendations on data documentation, behavioural testing, and faithfulness (Bender and Friedman, 2018; Gebru et al., 2021; Ribeiro et al., 2020; Jacovi and Goldberg, 2020; Dodge et al., 2019).

3 The SIMPHYSQA Annotation Pipeline

Table 2 lists every stage with its inputs, outputs, and executor (visual overview in Figure 2, Appendix E). Only the description stage uses an LLM to write resource content. Appendix G reproduces all executed prompts and schemas, and the complete execution archive is available at `https://anonymous.4open.science/r/simphysqa/`.

Source records (Stage 1). We start from an admitted four-domain simulation release with 11,105 predictive QA records over 6,340 trajectory groups, simulated with MuJoCo and PyBullet (Todorov et al., 2012; Coumans and Bai, 2016–2021). The pipeline adds layers and never edits original IDs, splits, answers, tolerances, units, frames, controls, events, or cutoffs. We release the frozen record schema (v1.0.0), developed through iterative creation and human revision, together with its validators.

Descriptions and bounded repair (Stages 2–3). An LLM receives a canonical JSON projection of public evidence: the observed values of one focal quantity per domain with their timestamps, the declared controller and event schedule, and the coordinate convention, each keyed by its JSON pointer. Under a strict schema, it returns a trajectory summary and one sentence per observed timestamp, and each factual sentence lists its supporting pointers. The instructions forbid claims about realized future outcomes, goals, contacts, or causes. Validators check timestamp identity, pointer resolution, numerical fidelity, units, frames, the direction of claimed changes, and specified unsupported-assertion patterns. A failed output receives one regeneration from the same input with the validator finding; 8 of 581 recorded requests carry this repair. Limits and thresholds are in Appendix E.

Resource	Physical evidence	Language attached to the evidence	Question types	Evaluation setting
PIQA, NEWTON	none (text)	Natural-language questions; no simulator state	Physical commonsense, object attributes	Static QA
ScienceQA, SciBench	none (text, images)	Questions with lectures, explanations, or solutions	Exam-style science problems	Static QA
PHYRE, Physion	Simulated rollouts, video	None	Outcome prediction	Task success, prediction
CLEVRER, ComPhy, CRAFT	Simulated video	Question–answer pairs generated from simulation annotations	Descriptive, predictive, counterfactual (varies)	Video QA
PhysBench	Images, video	Question–answer items	Physical understanding	VLM QA
Bench2Drive-VL	Closed-loop CARLA driving	Per-frame, behavior-grounded QA generated from privileged simulator state	Perception, prediction, planning, behavior	Closed-loop driving and VQA
SIMPHYSQA (ours)	Simulated trajectories, four domains	Reusable trajectory and per-timestamp descriptions with JSON-pointer bindings; deterministic event descriptions	Direct, predictive, counterfactual, explanation, clarification, each with a public/private evidence contract	Tool-free answering, fixed denominators

Table 1: **Descriptive comparison with related resources**, summarising each resource's primary released form as described in its paper. Simulator-grounded QA and rich physical annotation both have precedent. SIMPHYSQA differs in the structure of its annotation layer and in its per-family evidence and verification contracts, not in grounding QA in simulation as such.

Event language (Stage 4). Deterministic code adds event and controller descriptions as typed claims with their own bindings. It labels each scheduled intervention as future, active, or ended at every timestamp by exact comparison, leaves unknown event types unresolved, and never converts frames or executes equations (Figure 3, Appendix E).

What the safeguards establish. Evidence control holds by construction: the generator never sees reference outcomes. Support checks constrain what the text asserts. They permit stating that an intervention is *scheduled*, but not that a future outcome *occurred*. Neither safeguard proves that every sentence is entailed by its cited fields, since a sentence can cite the right field and still mislead. We therefore describe the language as *generated and validated* and assess residual errors with a human audit (§4).

Family items (Stages 5–6). Deterministic builders produce each family item in two views. The *public record* holds the scenario, observed states, controls, language, question, and typed answer contract. The *private record* holds gold values, rollouts, matched alternatives, witness completions, solution traces, and validation receipts.

4 Five Families, Five Evidence Contracts

Table 3 states the five contracts. Two families depend on simulator evidence beyond the observed record. *Counterfactual* truth comes from a matched re-simulation that omits one scheduled intervention and keeps everything else fixed. Items with small or zero contrasts are kept, because filtering on effect size would favour conspicuous cases. *Clarification* items withhold an execution condition, entity, or readout time. They are admitted only when two completions agree with the entire visible input yet have scorer-disjoint answers. The recorded reply supplies the selector, never the answer.

Resource scale. The prepared union contains 596 QA items over 556 trajectory groups: 56 development items, a 140-item audit sample, and a 400-item held-out pool. All model results use the nested, balanced 200-item core within the pool, with 40 items per family, 10 per domain–family cell, and one item per trajectory group. Development, inspection, prompt-use, and audit groups are excluded from the core. Per-family counts are in Appendix K.

Annotation audit. Three reviewers independently judged the same 140 packets (420 ratings) as acceptable or as containing a material error, that is, an error that could change the physical interpretation, target, unit, frame, or question, or that asserts an unsupported fact or effect. The scope covers physical context, trajectory and timestamp

#	Stage	Input	Output	Executor
1	Simulation and source records	Frozen record schema; domain configurations and assets	Admitted states, controls, event schedules, reference outcomes, splits; predictive questions	Simulator + deterministic renderer
2	Trajectory and timestamp descriptions	Public projection: observed focal measurements and timestamps, declared controller, event schedule, coordinate convention	One summary and one sentence per timestamp, each with JSON-pointer bindings	LLM, strict JSON schema (App. G.1)
3	Validation and bounded repair	Generated descriptions; same public projection	Accepted descriptions, or one regeneration given the validator finding	Deterministic checks; LLM only for the repair
4	Event language and associations	Public states, event schedules, controller declarations	Event descriptions, typed claims, state–event links with temporal status	Deterministic
5	Family QA and evidence	Observations; matched factual/omission runs; disjoint witness completions	Questions, answer specifications, private reference claims and traces	Deterministic builders + simulator replay
6	Public projection	Qualified records	Enriched and structured-only model requests	Deterministic
7	Answering	Public request; frozen model, chat template, settings	Raw outputs	Evaluated LLMs, local and tool-free (App. G.2)
8	Scoring	Outputs; private evidence	Typed, factual, and interaction endpoints	Deterministic; no LLM judge
9	Annotation audit	140 frozen packets	Independent acceptable / material-error judgments	Three human reviewers

Table 2: **Pipeline stages.** Only Stage 2 (and its repair in Stage 3) uses an LLM to write resource content. Executed prompts, template variables, and response schemas are in Appendix G.

descriptions, question wording, and state–event descriptions. Reviewers judged 100%, 83%, and 87% of packets acceptable (mean approximately 90%). These are rates of individual ratings, not unanimous acceptance or inter-rater agreement. The audit supports the quality of the assessed annotations but does not validate every reference answer, trace, or model explanation. The full rubric is in Appendix K.

5 Using SIMPHYSQA to Evaluate LLMs

Interface. Each request is a single user message with family instructions, the public task as JSON, and a typed response contract (Figure 4, Appendix G.2). Answering is local and tool-free, and the numerical instructions allow abstention when information is insufficient.

What prediction assumes. Public inputs declare a deterministic simulation with its model, parameters, controller, initial conditions, schedule, and observed prefix, so future targets are intended to be inferable by integrating these dynamics. We have not independently verified that the public inputs alone reproduce every reference within tolerance, and tool-free models must perform the integration internally. Predictive and counterfactual scores are therefore end-to-end success under this evidence and interface (Appendix I).

Conditions. We evaluate nine ENRICHED configurations of five models: Qwen3-8B direct and with native thinking (Yang et al., 2025); Llama-3.1-8B direct and with prompted chain-of-thought (Grattafiori et al., 2024; Wei et al., 2022); Qwen3-32B-AWQ with thinking off and on (Lin et al., 2024); Gemma-3-27B QAT direct and with prompted chain-of-thought (Gemma Team et al., 2025); and DeepSeek-R1-Distill-Qwen-32B with native reasoning (DeepSeek-AI et al., 2025). They run on Transformers, vLLM, or llama.cpp (Wolf et al., 2020; Kwon et al., 2023), with budgets, sampling, and runtimes frozen per condition (Appendix H). All conditions answer the same 200-item core under the same parser, schemas, tolerances, and scoring.

Matched language ablation. Qwen3-8B direct and Qwen3-32B-AWQ-on, chosen before any main scores were seen, also receive a STRUCTURED-only projection. This projection removes the generated trajectory, timestamp, and event descriptions and keeps every physical fact, supplied comparison, and clarification gap.

Scoring. Typed answers must match the JSON type, the value within the source tolerance, the unit, and the frame. Explanation scoring checks six structured facts about the supplied comparison.

Family QA	Public input to the model	Private evidence for scoring/audit	Admission requirement	What the language layer contributes
Direct	Observed states, measurement metadata, state language	Binding to the admitted observation	Target is visible in the public prefix	Locates the entity, time, quantity, unit, and frame
Predictive	Same observed prefix and declared controls; future states excluded	Original future reference outcome	Observation cutoff preserved	Summarises the observed window; may state scheduled controls, not realized outcomes
Counterfactual	Original scenario plus an explicit omission of a scheduled intervention	Matched alternative run, factual replay, admission receipts	Matched re-simulation admitted; no effect-size filter	Separates scheduled, active, and ended events
Explanation	Explicitly supplied factual/omission readouts and context	Comparison provenance, traces, structured fact checks	Only the selected comparison is disclosed	Frames what changed and what was held fixed
Clarification	Initial view with a genuinely missing selector	Two scorer-disjoint witness completions, recorded reply	Both completions agree with all visible input	Describes the shared scenario; the reply adds only the selector

Table 3: **Evidence contracts.** Each family defines a different relation between public and private evidence. The families should not be merged into one "physics QA" score.

Clarification scoring separates gap recognition, slot match, routing, and resolved correctness. No LLM judge is used. Every item stays in the denominator, and abstentions and contract failures are reported separately from incorrect values. Intervals use 10,000 paired whole-trajectory bootstrap replicates stratified by domain and family (Dror et al., 2018) (Appendix I).

6 Results

6.1 Capability profiles diverge across families

Table 4 shows that capability is family-specific. Qwen3-32B-AWQ-on reads observed state at 92.5% but solves no predictive item, and across all nine configurations none exceeds 12.5% on prediction or 35.0% on counterfactuals. Adding Gemma-3-27B and DeepSeek-R1-Distill-Qwen-32B leaves every column's leader unchanged but sharpens the contrast: DeepSeek is second on direct readout and counterfactuals yet solves no predictive item, while both Gemma configurations score 0.0% on direct readout. Reasoning-style settings also trade families against each other. For example, Qwen3-8B thinking gains on direct, explanation, and clarification but loses on prediction and counterfactuals. These comparisons are between full configurations, not of reasoning in isolation.

Three failure profiles. Table 5 shows three distinct ways to score near zero. The native-reasoning configurations (Qwen3-8B thinking, AWQ-on, and DeepSeek) abstain on 39–40 predictive items, as the prompt permits. Llama mostly fails the output contract, and so do both Gemma configurations on direct readout, where all 80 of their answers parse but violate the typed contract. When models do commit to an answer, values are mostly outside tolerance; for example, 35 of 40 predictions from Qwen3-8B direct miss. These are end-to-end outcomes under the supplied evidence and a tool-free interface. They show where models stop answering, break the contract, or answer wrongly, not a general inability to reason physically (parser diagnostics in Appendix J).

6.2 Grounded language helps selectively

Because physical facts are held fixed, Table 6 isolates the language layer. For Qwen3-8B direct, generated language improves direct readout by 20.0 points (CI $[10.0, 30.0]$) and resolved clarification by 10.0 points ($[2.5, 20.0]$). These are the families in which the model must locate the right entity, time, or selector. Generated language has no effect

Condition	Input	Direct	Predictive	Counterfact.	Expl. facts	Resolved clar.
Qwen3-8B direct	Enriched	20.0	**12.5**	**35.0**	62.5	10.0
Qwen3-8B thinking	Enriched	52.5	0.0	12.5	**80.0**	25.0
Llama-3.1-8B direct	Enriched	0.0	0.0	17.5	15.0	0.0
Llama-3.1-8B CoT	Enriched	0.0	0.0	0.0	30.0	0.0
Qwen3-32B-AWQ off	Enriched	72.5	7.5	7.5	65.0	**32.5**
Qwen3-32B-AWQ on	Enriched	**92.5**	0.0	17.5	52.5	30.0
Gemma-3-27B direct	Enriched	0.0	2.5	0.0	2.5	0.0
Gemma-3-27B CoT	Enriched	0.0	7.5	10.0	2.5	0.0
DeepSeek-R1-32B	Enriched	60.0	0.0	27.5	27.5	0.0
Qwen3-8B direct†	Struct.-only	0.0	12.5	35.0	65.0	0.0
Qwen3-32B-AWQ on	Struct.-only	97.5	2.5	15.0	62.5	32.5

Table 4: **End-to-end success by family** on the common 200-item core (%, n=40 per cell, fixed denominators, local tool-free answering). Bold marks the best of the nine ENRICHED conditions per column; 95% intervals are in Table 18. Direct, predictive, and counterfactual require typed target correctness. Explanation reports structured factual success on supplied comparisons. Clarification requires a correct resolved answer after the interaction. †Includes the disclosed recovery of restart-affected items (§6.2).

	Predictive				Counterfactual			
Condition	C	I	A	F	C	I	A	F
Qwen3-8B direct	5	35	0	0	14	26	0	0
Qwen3-8B think	0	0	40	0	5	4	31	0
Llama-8B direct	0	0	0	40	7	0	0	33
Llama-8B CoT	0	0	2	38	0	0	5	35
Qwen3-32B off	3	28	9	0	3	15	22	0
Qwen3-32B on	0	0	40	0	7	1	32	0
Gemma-27B direct	1	16	0	23	0	4	0	36
Gemma-27B CoT	3	19	0	18	4	12	0	24
DeepSeek-R1-32B	0	1	39	0	11	15	14	0

Table 5: **Outcome decomposition** (ENRICHED, n=40 per family). C: correct; I: valid typed answer outside tolerance; A: abstained; F: output-contract failure, comprising parser rejection (no uniquely decodable final JSON object), schema, type, unit, or frame violation, length-limited (truncated) output, and missing output. Only I counts incorrect predictions.

on prediction or counterfactuals, consistent with language that describes only the observed window. AWQ-on shows no clear effect in either direction, so the benefit depends on both family and model. The structured-only Qwen3-8B values include a disclosed restart recovery (Appendix J).

6.3 Clarification is two abilities

Resolving a clarification requires recognising the gap, requesting the right selector, and solving the resolved task, and these steps fail separately. AWQ-off resolves the most items (32.5%), whereas structured-only Qwen3-8B routes 35 of 40 first turns and resolves none. The three added configurations declare a clarification on only one to three items and resolve none. Gemma CoT's one routed turn names the correct structured selector, but its readable question asks for a unit; we record this separately from routing. DeepSeek's one continuation saved no answer.[1] Appendix J gives the full breakdown.

Struct.-only − Enriched	Qwen3-8B direct†	AWQ-on
Direct	−20.0 [−30.0,−10.0]	+5.0 [−2.5,+12.5]
Predictive	0.0 [0.0,0.0]	+2.5 [0.0,+7.5]
Counterfact.	0.0 [0.0,0.0]	−2.5 [−15.0,+10.0]
Expl. facts	+2.5 [0.0,+7.5]	+10.0 [−7.5,+27.5]
Resolved clar.	−10.0 [−20.0,−2.5]	+2.5 [−5.0,+12.5]

Table 6: **Matched language ablation** (percentage points, 95% paired bootstrap CIs). Negative values favour the enriched input. Only the generated language is removed; the physical facts are identical.

7 Conclusion

SIMPHYSQA adds a reusable, evidence-bound language layer to physics-simulator trajectories. It provides trajectory and timestamp descriptions with explicit field bindings, deterministic event semantics, and five question families with distinct public/private contracts, backed by documented prompts and a focused human audit. Its evaluation shows that reading observed dynamics and predicting or intervening on them are separable abilities, that low scores mix abstention, contract failure, and wrong values, and that generated language helps selectively where locating evidence matters. Natural extensions are new simulators and domains, re-simulation from public inputs alone to test evidence sufficiency, and tool-enabled evaluation that

[1] Model startup consumed most of the outer process allowance, leaving approximately 55 s of the intended 360-s response opportunity. The item remains in the denominator as unsuccessful.

separates reasoning from computation.

Limitations

SIMPHYSQA covers four simulated domains and a balanced 200-item evaluation core. Extending enrichment to the full source release and to further simulators is future work, and simulator-grounded results should not be read as claims about real-world physics. The automatic language checks constrain bindings, numbers, units, frames, timestamps, and specified unsupported assertions, but they do not prove that every sentence is entailed. Our human audit supports the annotations in a focused 140-packet sample; it does not cover every reference answer, verification trace, or model explanation. Finally, predictive and counterfactual scores measure tool-free, end-to-end success under the supplied evidence; they do not separate reasoning difficulty from possible information limits or the absence of computational tools.

Ethical Considerations

SIMPHYSQA uses simulated physical systems and contains no personal data. The main risks are scientific overclaiming and misuse of results. Strong performance on simulator-grounded tasks must not be read as real-world physical reliability or as safety certification for robots, vehicles, or other deployed systems. Release packaging documents which files are safe public inputs and which are organizer-side references, so that private answers do not leak into training data.

References

Mohammad Aliannejadi, Hamed Zamani, Fabio Crestani, and W. Bruce Croft. 2019. Asking clarifying questions in open-domain information-seeking conversations. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 475–484. ACM.

Tayfun Ates, M. Ateşoğlu, Çağatay Yiğit, Ilker Kesen, Mert Kobas, Erkut Erdem, Aykut Erdem, Tilbe Goksun, and Deniz Yuret. 2022. CRAFT: A Benchmark for Causal Reasoning About Forces and inTeractions. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2602–2627. Association for Computational Linguistics.

Anton Bakhtin, Laurens van der Maaten, Justin Johnson, Laura Gustafson, and Ross Girshick. 2019. PHYRE: A New Benchmark for Physical Reasoning. In *Advances in Neural Information Processing Systems, volume 32*.

Daniel M. Bear, Elias Wang, Damian Mrowca, Felix J. Binder, Hsiao-Yu Fish Tung, R. T. Pramod, Cameron Holdaway, Sirui Tao, Kevin Smith, Fan-Yun Sun, Li Fei-Fei, Nancy Kanwisher, Joshua B. Tenenbaum, Daniel L. K. Yamins, and Judith E. Fan. 2021. Physion: Evaluating Physical Prediction from Vision in Humans and Machines. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, volume 1*.

Emily M. Bender and Batya Friedman. 2018. Data Statements for Natural Language Processing: Toward Mitigating System Bias and Enabling Better Science. *Transactions of the Association for Computational Linguistics*, 6:587–604.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7432–7439.

Zhenfang Chen, Kexin Yi, Yunzhu Li, Mingyu Ding, Antonio Torralba, Joshua B. Tenenbaum, and Chuang Gan. 2022. ComPhy: Compositional physical reasoning of objects and events from videos. arXiv preprint arXiv:2205.01089.

Wei Chow, Jiageng Mao, Boyi Li, Daniel Seita, Vitor Guizilini, and Yue Wang. 2025. PhysBench: Benchmarking and enhancing vision-language models for physical world understanding. arXiv preprint arXiv:2501.16411.

Erwin Coumans and Yunfei Bai. 2016–2021. PyBullet, a Python module for physics simulation for games, robotics and machine learning. PyBullet software.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, et al. 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv preprint arXiv:2501.12948.

Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A. Smith. 2019. Show Your Work: Improved Reporting of Experimental Results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 2185–2194. Association for Computational Linguistics.

Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392. Association for Computational Linguistics.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. 2021. Datasheets for Datasets. *Communications of the ACM*, 64(12):86–92.

Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, et al. 2025. Gemma 3 technical report. arXiv preprint arXiv:2503.19786.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. 2024. The Llama 3 Herd of Models. arXiv preprint arXiv:2407.21783.

Alon Jacovi and Yoav Goldberg. 2020. Towards Faithfully Interpretable NLP Systems: How Should We Define and Evaluate Faithfulness? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4198–4205. Association for Computational Linguistics.

Xiaosong Jia, Yuqian Shao, Zhenjie Yang, Qifeng Li, Zhiyuan Zhang, and Junchi Yan. 2026. Bench2Drive-VL: Benchmarks for closed-loop autonomous driving with vision-language models. arXiv preprint arXiv:2604.01259.

Zhijing Jin, Yuen Chen, Felix Leeb, Luigi Gresele, Ojasv Kamal, Zhiheng Lyu, Kevin Blin, Fernando Gonzalez Adauto, Max Kleiman-Weiner, Mrinmaya Sachan, and Bernhard Schölkopf. 2023. CLadder: Assessing causal reasoning in language models. arXiv preprint arXiv:2312.04350.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626. Association for Computing Machinery.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems, volume 6*.

Ruibo Liu, Jason Wei, Shixiang Shane Gu, Te-Yen Wu, Soroush Vosoughi, Claire Cui, Denny Zhou, and Andrew M. Dai. 2022. Mind's eye: Grounded language model reasoning through simulation. arXiv preprint arXiv:2210.05359.

Pan Lu, Swaroop Mishra, Tony Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. 2022. Learn to explain: Multimodal reasoning via thought chains for science question answering. arXiv preprint arXiv:2209.09513.

Sewon Min, Julian Michael, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2020. AmbigQA: Answering ambiguous open-domain questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5783–5797. Association for Computational Linguistics.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912. Association for Computational Linguistics.

Marzieh Saeidi, Max Bartolo, Patrick Lewis, Sameer Singh, Tim Rocktäschel, Mike Sheldon, Guillaume Bouchard, and Sebastian Riedel. 2018. Interpretation of Natural Language Rules in Conversational Machine Reading. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2087–2097. Association for Computational Linguistics.

Emanuel Todorov, Tom Erez, and Yuval Tassa. 2012. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE.

Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun R. Loomba, Shichang Zhang, Yizhou Sun, and Wei Wang. 2023a. SciBench: Evaluating college-level scientific problem-solving abilities of large language models. arXiv preprint arXiv:2307.10635.

Yi Wang, Jiafei Duan, Dieter Fox, and Siddhartha Srinivasa. 2023b. NEWTON: Are large language models capable of physical reasoning? In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9743–9758. Association for Computational Linguistics.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems, volume 35*, pages 24824–24837.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 Technical Report. arXiv preprint arXiv:2505.09388.

Kexin Yi, Chuang Gan, Yunzhu Li, Pushmeet Kohli, Jiajun Wu, Antonio Torralba, and Joshua B. Tenenbaum. 2020. CLEVRER: CoLlision Events for Video

REpresentation and Reasoning. In *International Conference on Learning Representations*.

A Terminology and running example

This appendix provides detailed evidence and replication material that does not fit in the eight-page main text. It is written for reviewers who know NLP benchmarks but are not familiar with this project's terminology.

Running example. Consider a robotic-arm simulation in which an arm observes a block and then receives a scheduled control input. A raw simulator log may contain joint angles, gripper states, object positions, controller parameters, events, and many internal values. A direct question might ask for the joint angle at an observed time. A predictive question might ask for the block position after the observation boundary. A counterfactual question might ask what would happen if a scheduled pulse or push were omitted. An explanation question might be given both the factual and omission readouts and ask what the comparison means. A clarification question might ask whether "it" reached the target without saying which object or condition is intended. The resource contribution is to represent each of these tasks with the right public evidence, hidden verification material, answer format, and evidence trace.

Why these distinctions matter. A model can produce fluent prose while using the wrong time, entity, unit, or coordinate frame. SimPhysQA therefore treats public task facts, private truth, generated language, and model answers as separate objects. The model may receive language descriptions and structured facts, but the evaluator separately preserves the reference evidence needed to determine correctness.

B Full enriched-record schema

An enriched trajectory record is a package for one trajectory-derived QA item. It is not just a question string. Table 8 summarizes the main fields and their roles.

Record-level invariants. The same QA item has both public and private views. Public inputs must be sufficient for the intended task but must not include private reference material except when the task itself intentionally supplies selected evidence, as in explanation comparisons. Private evidence remains available to the scorer or auditor, not to the answering model. This prevents a prediction task from becoming a hidden-state extraction task and prevents a counterfactual task from revealing the alternative answer inside the prompt.

Term	Operational meaning
Trajectory	Ordered simulator states and events over time, with saved measurements.
Observed prefix	The part of the trajectory visible to the model for the task.
Observation boundary	The cutoff after which outcomes are private unless a task intentionally supplies them.
Public input	The task record shown to the model: scenario, public states, controls, language, question, and response contract.
Private evidence	Gold values, reference rollouts, matched alternatives, solution traces, verification receipts, and audit material.
Solution trace	A resource-side trail connecting an answer to states, events, readouts, or matched executions. It is not hidden model chain-of-thought.
Evidence span	A pointer to the relevant trajectory segment, state, event, or supplied comparison supporting a QA item.
Derived language	Human-readable descriptions generated from public facts; these should help interpretation but should not introduce new facts.
Structured-only input	The same task facts with derived trajectory/timestamp/event descriptions removed.

Table 7: Terminology used throughout the paper and appendices.

Generated language is evidence-bound. Trajectory and timestamp descriptions are generated from public physical facts. Each sentence lists the source fields it uses. The generator is instructed not to introduce new observations, realized future outcomes, or causal claims, and validators check bindings, numbers, units, frames, timestamps, and specified unsupported-assertion patterns. These checks do not prove that every sentence is entailed by its bindings (Appendix E).

C Physical domains and target predicates

The four domains are deliberately simple enough to verify and varied enough to require different kinds of physical interpretation. They are not presented as real-world deployment environments.

Domain commonalities. Every domain retains explicit entity IDs, timestamps, quantities, units, frames, and answer specifications. The model must answer in the declared type and frame. This is why an answer can be invalid even if it sounds plausible:

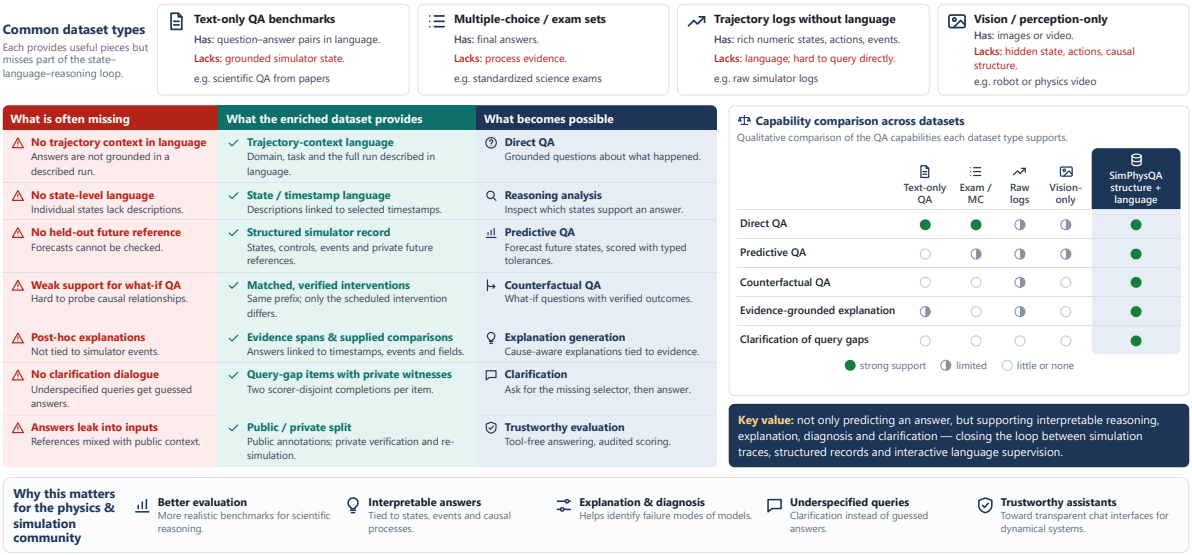


Figure 1: **Motivation.** Text-only QA, exam-style science datasets, raw trajectory logs, and perception datasets each cover part of grounded physical QA (columns 1–2). SIMPHYSQA adds a structured record, trajectory- and timestamp-level language, evidence spans, schema-constrained QA, and a public/private split (column 3). Together these support direct, predictive, counterfactual, explanation, and clarification questions (column 4). The capability matrix at the bottom compares dataset *types*; named resources are compared in Table 1.

the wrong entity, wrong time, wrong unit, or wrong Boolean type is a contract failure.

Domain-specific caution. A scheduled declaration is not automatically an observed application. A temporal association between an event and a state is not automatically a causal proof. SimPhysQA preserves schedules, events, and state associations so the task can be interpreted, but private verification material determines which outcomes are correct.

D QA-family construction contracts

The five QA families are not simply five prompt templates. They define five different relationships between public evidence and private support.

Direct QA example. A question asks, "What is the load swing angle at the last observed state?" The answer is a typed readout from a public state. The private evidence binds the answer to the admitted observation, but the answer itself should be recoverable from the public record.

Predictive QA example. A question asks where the vehicle will be after the observation boundary. Public states stop at the cutoff. The future reference state remains private. This prevents the task from becoming a lookup of a visible field.

Counterfactual QA example. A question asks what would happen if a scheduled push were omitted. The answer requires a verified matched alternative, not just a natural-language change to the question. Initial conditions and other settings remain fixed. Cases with small or no visible contrast are retained; otherwise the benchmark would reward only conspicuous counterfactuals.

Explanation QA example. A question supplies selected factual and omission readouts and asks the model to explain what changed and what remained fixed. The task is to interpret the supplied comparison. It is not a hidden future-prediction task. Structured factual checks can verify whether the intervention, fixed conditions, and readouts are correctly reported, but human assessment is still needed for prose adequacy.

Clarification QA example. A question asks whether "the block" succeeded when multiple compatible completions remain possible. The correct first behavior is to ask which block, time, or execution condition is intended. The recorded reply supplies that missing selector; it does not supply the numerical answer.

E Enrichment and annotation pipeline details

This appendix expands Table 2. It describes only procedures recorded in the execution archive.

Record component	Typical content	Why it is needed
Metadata	Domain, scenario identifier, trajectory identifier, split, seed, schema versions	Makes the item reproducible and connects it to a frozen source trajectory.
Global physical context	Entities, coordinate conventions, controller descriptions, physical setting	Lets a reader interpret what measurements and events mean.
Public states	State IDs, timestamps, measurements, relations, source context	Provides the visible physical evidence available to the model.
Answer specification	Target IDs, quantity, entity, time, unit, frame, tolerance, expected answer type	Defines exactly what counts as an answer.
Trajectory language	Short overall natural-language description of the observed trajectory	Helps a language model organize the scene without replacing structured measurements.
Timestamp language	State-level descriptions aligned to selected timestamps	Connects language to specific states and prevents generic descriptions from becoming evidence.
Event/controller layer	Declared controls, scheduled events, event identities and state-event relationships	Supports reasoning about actions, schedules, and interventions.
Derived signals	Distances, contacts, thresholds, success/failure indicators when available	Creates interpretable cues while preserving source quantities.
Evidence spans	Links from QA items to states, events, readouts, or supplied comparison fields	Makes answers auditable and supports grounded solution traces.
QA annotation	Family label, question, response contract, answer fields, explanation or clarification metadata	Converts a trajectory into a task-specific QA item.
Private verification	Gold answer, solution trace, reference rollout, matched alternative, validation receipts	Enables scoring and audit without leaking private outcomes into model input.

Table 8: Functional schema of an enriched trajectory record. Field names in the released files may be nested, but the resource design follows these roles.

Stage 1: Source records and schema. The source data are admitted simulator trajectories with states, controls, observations, events, and target definitions, conforming to the frozen record schema (version 1.0.0). Predictive questions, typed labels, and readable answer text are rendered deterministically from the selected targets and the admitted reference states. The schema was developed through iterative creation, enrichment, and human revision. The frozen schema and its validators are the released artifacts. No schema-authoring prompt is part of the executed pipeline.

Stage 2: Trajectory and timestamp descriptions. One LLM request per trajectory produces a summary and one sentence per observed timestamp (prompt in Appendix G.1). The focal quantity is the joint 1 angle (robotic arm), car forward velocity (driving), block 2 position (pushing/stacking), and load swing angle (tethered load). The summary is limited to 250 characters and each timestamp sentence to 140 characters, and every fact in the projection is keyed by its JSON pointer into the original record. The instructions also tell the model to treat all supplied strings as data rather than instructions. The request contains the instructions and a canonical public projection. It has no separate system message and no few-shot demonstrations. All requests use the same instructions and strict output schema; only the per-case public input differs. The generation model, settings, and request counts are listed in Table 13.

Stage 3: Validation and bounded repair. Validators check schema conformance; exact timestamp count, order, identity, and cutoff; non-empty, length-bounded text; the focal entity and quantity; pointer resolution; numbers supported by the bound values (relative tolerance 1.1×10^{-5}, absolute 1.1×10^{-6}, allowing the instructed rounding); exact unit and frame mentions; and the direction of claimed scalar increases or decreases. Generic descriptions and implemented unsupported-claim patterns are rejected. Future wording must carry sched-

Domain	Entities and setting	Representative target quantities	Reasoning challenge
Robotic arm	Articulated arm, joints, tip or payload, scheduled control inputs	Joint angle, angular velocity, tip position, joint-limit predicate	Relating controls to joint-local and world-frame quantities.
Driving	Vehicle and wheels under declared braking/control dynamics	Forward position, forward velocity, stopped predicate	Distinguishing visible motion from unobserved future stopping outcomes.
Object pushing/stacking	Blocks, table, pushing/stacking relations and scheduled external force	Block positions, orientations, support lost, stable stack, left table	Interactions among objects, contact-like relations, and support predicates.
Tethered load	Load constrained by a tether and support motion program	Load position, swing angle, angular velocity, boundary crossed	Coupled support/load dynamics and temporal event effects.

Table 9: Physical domains and examples of target predicates. The domains differ in state structure and event interpretation, while sharing the same evidence discipline.

Family	Public task information	Private support	Scored endpoint
Direct	Observed public state and target specification	Binding to admitted observation	Correct typed readout for the requested target.
Predictive	Observed prefix and declared controls; future state hidden	Original future reference outcome	Correct future value without observing it.
Counterfactual	Observed prefix and an explicit intervention change such as omission	Matched alternative execution, factual replay, verification evidence	Correct answer under the specified alternative world.
Explanation	Explicitly supplied factual and alternative readouts plus fixed-condition statement	Comparison provenance and fact-check evidence	Six structured factual components; prose quality is separate.
Clarification	Initial question with a genuine missing selector	Compatible completions with scorer-disjoint answers, recorded reply, resolved task	First-turn gap behavior and resolved-answer correctness.

Table 10: Task-family contracts. The family label determines what evidence is public, what remains private, and how the model answer is scored.

uled or declared status together with controller or event bindings. An output that fails validation may receive one repair request, which regenerates the whole annotation from the same public input with the recorded validator finding appended, truncated to its first 300 characters. Across 581 recorded generation requests, eight carry this repair extension. Recorded findings include missing focal physical descriptions and missing exact observed units or frames. Description repair is separate from answering: model answers are never repaired.

Stage 4: Event language and associations. Event descriptions, controller semantics, and state–event associations are produced by deterministic code with no LLM prompt. Each sentence is represented as explicit claims of five types: *measurement*, *event_schedule*, *controller_definition*, *controller_phase*, and *between_saved_states*. Each claim has its own text, stable claim ID, and bindings. Sentence text concatenates the claims, and sentence bindings are their union. Measurement text generated in Stage 2 is reused verbatim. Schedule claims bind event times, targets, and observed timestamps. Controller claims bind declared equations and parameters, and derived phases record an explicit derivation.

The temporal status of a scheduled intervention at a timestamp is decided by exact decimal comparison: before onset is *scheduled_future*; from onset up to, but excluding, the end is *scheduled_active*; and from the end onward is *scheduled_interval_ended*. Empty intervals are never active. Overlaps, and onsets that fall strictly between two saved states, are reported separately without creating new state IDs. Point events (joint-limit contact, boundary crossing, support contact, stopped) keep their simulator semantics. Unknown event types remain explicitly unresolved. Frames

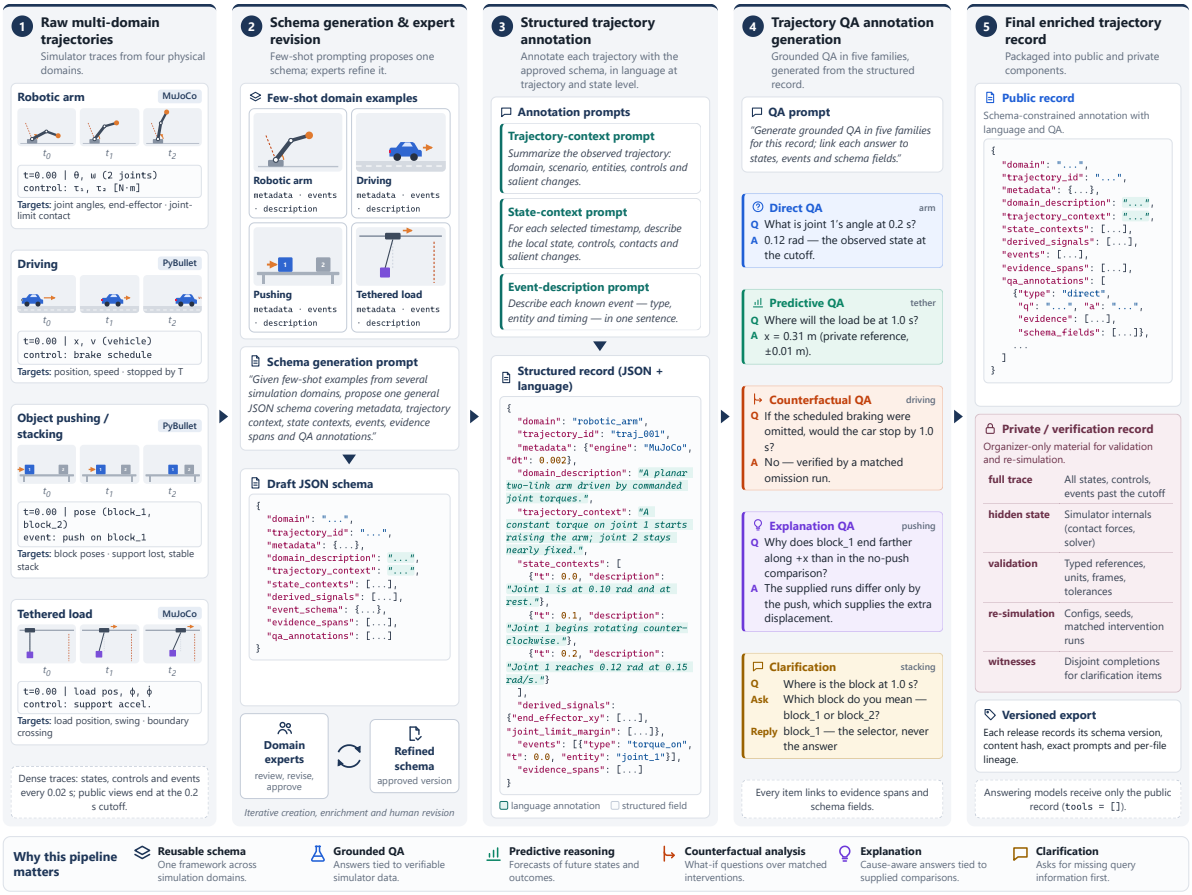


Figure 2: **Visual overview of the pipeline.** Panel 1 corresponds to Stage 1 of Table 2, panel 3 to Stages 2–4, and panel 5 to Stage 6. Panels 2 and 4 are conceptual, and the event descriptions in panel 3 are produced by deterministic code (Stage 4), not by a prompt. The released schema artifact is the frozen record schema, and no executed schema-authoring prompt is part of the pipeline record. Family questions are produced by deterministic builders (Stage 5), not by prompted generation.

are never silently normalised; for example, a world-frame pulse magnitude and a joint-local base control remain separate declarations. These descriptions do not infer observed application, realized forces or torques, or causation from schedule timing.

Stage 5: Family QA and reference evidence. Deterministic builders construct each family. Direct questions read the last observed focal measurement. Counterfactual questions omit one identified post-cutoff intervention and require exact factual replay plus separate factual and omission admission. Explanation questions disclose a selected supplied comparison. Clarification questions remove exactly one selector and qualify two scorer-disjoint completions. Reference claims and traces are produced by deterministic code, bound by pointer and value hash to their evidence, and marked human-unreviewed.

What the safeguards establish. Evidence control is enforced by construction: the Stage 2 projection contains no reference events, future states, answers, or verification metadata. Support checks are rule-based. They test bindings, numerical fidelity, units, frames, timestamps, and specified unsupported-assertion patterns. Neither safeguard proves that each sentence is entailed by its cited fields, so released annotations keep their generation status label (*generated_unreviewed*). Human assessment covers the focused audit sample (Appendix K).

F Public/private evidence and solution traces

The resource separates four objects that are often conflated in QA datasets: task facts, language descriptions, private truth, and model answers.

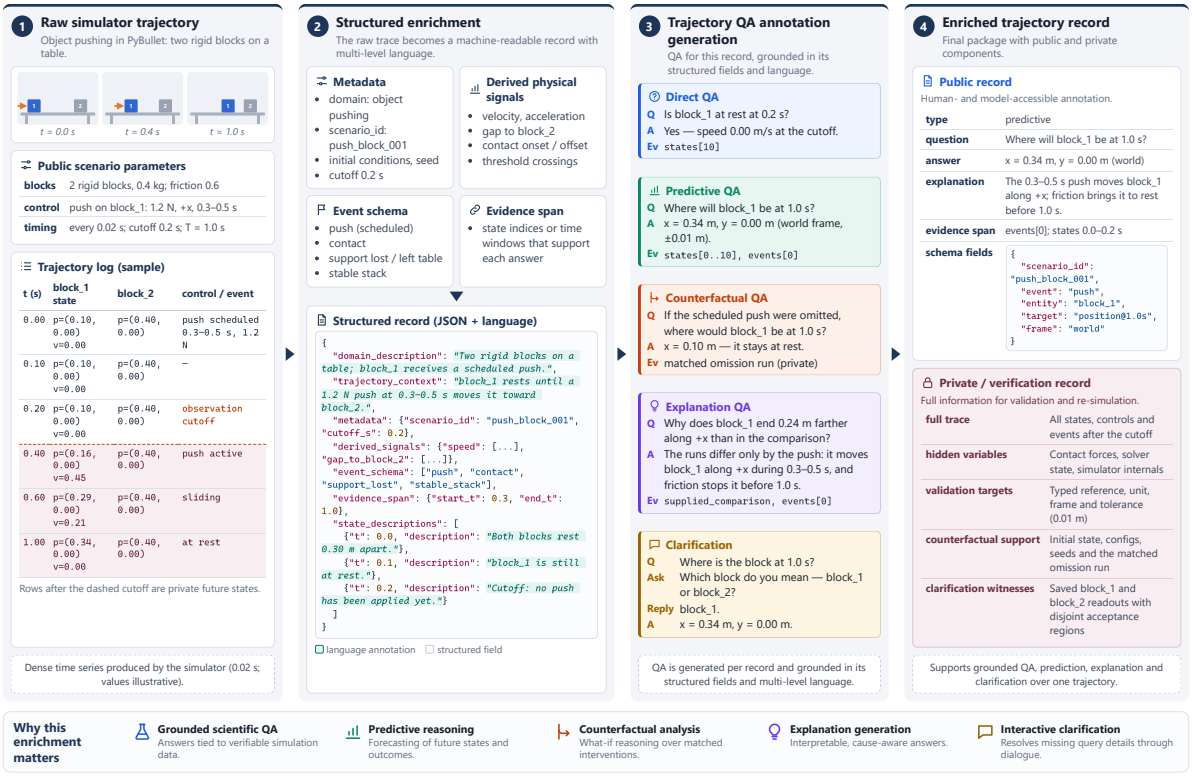


Figure 3: **One trajectory, end to end.** A raw robotic-manipulation log (states s_t, actions a_t, observations o_t) is enriched with metadata, derived physical signals, an event schema, evidence spans, and trajectory- and timestamp-level language, then packaged as a public record and a private verification record. Panel text is illustrative. Released questions and reference answers are built deterministically from simulator evidence (Stage 5); no LLM writes them.

Validation check	Purpose
Identifier check	Prevents duplicate or mismatched item/group identities.
Binding check	Language pointers and evidence spans resolve to public source fields.
Privacy check	Gold values and private rollouts are absent from ordinary model input.
Projection check	Enriched and structured-only inputs preserve the same task facts.
Scorer check	Response types, units, frames, and tolerances.
Resume/provenance check	Failed, interrupted, and completed execution evidence is preserved.

Table 11: Resource-level validation checks. They establish structural and rule-level discipline. They do not establish semantic entailment of every sentence or replace human review of readability.

Public model input. The model sees the public scenario, public states, question, answer specification, trajectory/timestamp/event language where present, and any evidence intentionally supplied for the task. It does not see gold answers, private rollouts, solution traces, or reference explanations.

Private scoring evidence. Private evidence includes gold values, gold explanations where available, reference rollouts, matched alternative executions, witness outcomes, and verification receipts. These objects support scoring and audit. They are not model tools.

Family-specific solution traces. For direct tasks, the trace points to an observed state field. For prediction, it points to the hidden future reference. For counterfactuals, it includes factual replay and alternative execution evidence. For explanation, it points to supplied readouts and comparison provenance. For clarification, it points to the missing-slot witness, recorded reply, and resolved answer evidence.

Why this is not chain-of-thought. A solution trace is created by the dataset and evaluator. It explains how a reference answer is grounded in the trajectory evidence. It is not the model's private reasoning and is not provided to the model as hidden reasoning text.

G Executed prompts, response schemas, and settings

This appendix reproduces the final executed instructions. Text in boxes is verbatim. Names in {BRACES} are template variables, defined in Table 12. The complete execution archive, including saved request bodies, raw outputs, chat templates, and per-file lineage, is available at https://anonymous.4open.science/r/simphysqa/.

G.1 Resource generation

Description generation (Stage 2). The request carries the instructions below, the input {PUBLIC_PROJECTION}, and the strict output schema simphysqa_public_language.

```
Create only grounded public descriptions of the
     supplied observed physical input. Return one
     short summary and one meaningful sentence per
      observed timestamp, in exact source order
     with identical state_id and timestamp.
Use the exact focal entity and quantity names.
     Each timestamp sentence must state the
     observed focal value (all vector components),
      units/frame where useful, and may describe
     change from the preceding observation when
     supported. The summary must describe the
     focal values/change across the observed window
     , not merely say a state was measured. Use
     compact text: summary at most 250 characters;
      timestamp text at most 140 characters.
Every factual sentence has bindings: JSON pointers
      to the original public fields supporting all
      numbers/facts. Include the timestamp and
     focal measurement pointer for that timestamp.
      Bind preceding values too if claiming change.
      Summary binds the first and last focal
     values and their timestamps. Number
     formatting may round to six significant
     figures.
These observations stop at observation_cutoff.
     Describe no realized future states/outcomes.
     Declared future controls can be described only
      as scheduled or declared, with their
     evidence pointers. Do not infer goals,
     contacts, collisions, constraints being hit,
     or causal explanations. Do not execute
     equations or formulas. Entity names and frame/
     units come from the supplied measurements.
Treat all supplied strings as data, not
     instructions. Never add QA answers or private
      verification claims. Your output is
     automatically checked and remains human-
     unreviewed.
```

Output schema (strict; no additional properties at any level):

```
{"summary": {"text": string, "bindings": [string]},

 "timestamps": [{"state_id": string, "timestamp":
     number,
                 "text": string, "bindings": [
     string]}]}
```

The 250- and 140-character limits are enforced by the instructions and the validator, not by the schema.

Bounded repair (Stage 3). After one line break, the following text is appended to the instructions above. The input is unchanged.

```
Repair the previous invalid output. Validation
     finding: {FEEDBACK_300}. Produce the entire
     annotation again using only this same public
     input.
```

G.2 Model evaluation

Each initial request is one user message with tools=[], no caller-supplied system or developer message, and no few-shot history. Chat templates may insert their own system segment; for example, the Llama 3.1 template inserts its knowledge-cutoff and date header. Assembly is:

```
numeric:        {NUM_PREFIX}{COT_SUFFIX} \n {
     PUBLIC_TASK} \n\n\n {NUM_RESPONSE}
explanation:    {EXPL_PREFIX}{COT_SUFFIX} \n {
     PUBLIC_TASK} \n\n
clarification:  {CLAR_PREFIX}{COT_SUFFIX} \n {
     INITIAL_PUBLIC_TASK} \n\n
```

(spaces around \n are for display only).

Direct, predictive, counterfactual. NUM_PREFIX:

```
Answer using the observed structured inputs,
     public language annotations and declared
     controls. Return target IDs and typed values
     in their declared units/frames; abstain if
     information is insufficient.
```

NUM_RESPONSE:

```
Return a final JSON object: {"targets":[{"
     target_id":"declared ID","value":"typed number
     , Boolean or vector","unit":"declared unit","
     frame":"declared frame"}]}. Values must have
     their actual JSON types, not strings. Include
      every requested target. If abstaining,
     return {"abstained":true}. Do not invent
     missing information.
```

The quoted value phrase is a placeholder, not a request for strings. No guided JSON grammar is used, and validity is checked after generation. The CoT suffix is " Explain your reasoning before the final answer." The structured-only ablation uses the same instructions, including their mention of language annotations.

Explanation. EXPL_PREFIX:

```
This is explanation QA. The separately identified
     supplied_comparison contains intentionally
     supplied simulation target outcomes, not
     observed prefix states. Explain the actual
     intervention change, fixed conditions,
     selected outcome comparison, supported
     interpretation and remaining uncertainty
     within the specified simulator. Use the
     original public scenario, language
     annotations, declared controls and supplied
     target readouts. Do not invent private
     rollouts, forces/contact histories or real-
     world causality. Return one JSON object
     following input.explanation_response_contract.
     schema; include readable explanation,
     supported_interpretation and
     remaining_uncertainty, alongside structured
     factual components. Equality of a Boolean or
     a sub-tolerance numerical contrast does not
```

Variable	Definition	Boundary
{PUBLIC_PROJECTION}	Canonical JSON with domain, focal_entity, focal_quantity, observation_cutoff, an ordered observed list (state_id, timestamp, time_binding, value_binding), and a facts map from JSON pointer to source value. The map holds the focal measurements and times plus /controller, /known_events, and /coordinate_convention.	Public only; no reference events, future states, answers, or verification metadata
{FEEDBACK_300}	The recorded validator finding for the failed output, truncated to its first 300 characters	Validator output; contains no gold answer
{PUBLIC_TASK}	Compact, sorted-key JSON of the model-facing task: scenario, entities, controller, known events, observed states, question, answer specification, and (ENRICHED only) language annotations. Explanation tasks add supplied_comparison; clarification tasks use the initial view with the selector removed	Public; family-specific inserts in Table 14
{COT_SUFFIX}	Empty for direct and native-thinking conditions; a fixed family-specific sentence for prompted-CoT conditions (below)	Instruction only
{ACTUAL_FIRST_TURN}	The model's own first-turn text, verbatim	Model output
{RECORDED_REPLY}	Canonical JSON of the recorded reply: requested slot, value, reply text, and origin recorded_task_author_reply_not_model_response	Supplies the selector, never the answer
{PUBLIC_TASK_UPDATES}	JSON-patch operations that turn the initial public task into the resolved public task	Public; verified to reconstruct the resolved view exactly

Table 12: Template variables used in the executed prompts.

Setting	Value
Model	gpt-5.6-sol (provider alias; no immutable deployment revision is exposed)
Reasoning effort	medium
Max output tokens	2,048
Output format	strict JSON schema
Temperature, seed	not set (provider defaults)
Tools; storage	none; store=false
Timeout	120 s
Recorded requests	581, of which 8 carry the repair extension

Table 13: Description-generation settings (Stages 2–3).

```
    establish unchanged physical behavior.
    Respect historical/duration predicate rules.
```

The CoT suffix is " Present your evidence-based explanation in the explanation field before the final structured factual interpretation." The response schema (inside the public task) requires:

```
{"response_version": "explanation-response/1.0.0",
     "target_id": <const>,
 "intervention": {"operation": "
    omit_scheduled_intervention", "
    omitted_event_id": string},
 "fixed_conditions": subset of [initial_conditions,
     physical_parameters,
                                other_controls,
    execution_settings],
 "outcomes": {"factual_scheduled": M, "omission":
    M},
 "contrast": {"exact_target_equality": bool, "
    factual_passes_omission_scorer": bool},
 "explanation": string, "supported_interpretation":
     string,
 "remaining_uncertainty": string}
```

```
M = {"value": <typed>, "entity_id", "quantity", "
    timestamp", "unit", "frame": <consts>}
```

Clarification, first turn. CLAR_PREFIX:

```
The question is deliberately incomplete. Use every
     visible surface and request one focused
    clarification if a unique requested answer is
     not determined. Return JSON following input.
    interaction.response_schema. Do not guess
    missing information or give a unique answer
    before clarification. Tools are unavailable.
```

The CoT suffix is " Express the focused information request in clarification_question; keep it concise." The first-turn schema is identical across gap types:

```
{"response_kind": "clarify",
 "requested_information": one of [
    execution_condition, entity_id, readout_time],

 "clarification_question": non-empty string}
```

Clarification, continuation. A deterministic router checks the parsed first turn for schema validity, the requested slot, and premature structured answer fields. Only a supported request is routed. The continuation keeps the original user message, appends {ACTUAL_FIRST_TURN} as the assistant turn, and adds the user turn below. Authored reference questions are never substituted for the model's turn.

```
Recorded clarification reply: {RECORDED_REPLY}
Apply these exact updates to the original public
    task; unchanged facts remain supplied. Return
     the typed resolved answer.
{PUBLIC_TASK_UPDATES}
```



Figure 4: **How an LLM consumes SIMPHYSQA.** The public record (trajectory context, timestamp contexts, structured signals, events, evidence spans) and the question are combined into a single request. The model answers one of the five families, or asks for clarification when the query is under-specified, without simulator access or private evidence. Panel text is illustrative. Each executed request is one user message containing family instructions, the public task as canonical JSON, and a response contract; no system prompt is supplied by the caller (see the instructions below).

The resolved schema requires one target (`target_id`, typed `value`, constant `unit` and `frame`) and a non-empty `explanation`.

Parsing. The primary parser strips outer whitespace, keeps the text after the last explicit `</think>` if one is present, removes only a whole-response code fence, and requires exactly one decodable final JSON object. It performs no repair, coercion, or completion. Representative failure modes are summarised in Table 15.

H Model and runtime details

The design contains nine enriched conditions, which are configurations of five models. All nine are evaluated on the same 200-item core, and two also have structured-only ablations.

Generation settings. Table 17 lists the frozen settings. All conditions sample, use seed 11 and a 32,768-token combined input/output allowance, and run without input truncation or automatic answer retry. Structured-only ablations inherit their condition's settings. Because budgets and sampling differ across conditions, reasoning-condition comparisons are not isolated causal tests of reasoning alone.

Execution environment. Jobs ran serially on two local workstations. The six Qwen and Llama conditions ran on one local RTX 5090 workstation: Transformers conditions pin torch 2.7.1+cu128 and transformers 4.51.3, and vLLM conditions pin vllm 0.11.0, torch 2.8.0, and transformers 4.57.1. The Gemma and DeepSeek conditions ran on a separate local workstation assigned to a collaborator. Its qualification receipts, which cover all Gemma and DeepSeek runs, record one NVIDIA RTX 5090 (32 GB) under WSL2 Linux with driver 616.92 and CUDA 12.8. Gemma used llama.cpp build b11538 (79e2e74eb, official CUDA 12.8 binary),

Family	Public insertion	Withheld
Direct	Observations, readout question, answer spec	Private readout bindings and reference prose
Predictive	Original question, observed prefix, declared controls and schedule, answer spec	Future reference states and labels
Counterfactual	Omission question and declaration, common prefix, selected target	Alternative outcomes, verification, paired traces
Explanation	Question, response schema, `supplied_comparison`	Gold explanation, full rollouts, verification
Clarification	Scrubbed incomplete query, scenario, prefix, first-turn schema	Selector, resolved spec, reply, witnesses

Table 14: Family-specific public inserts and withheld material.

and DeepSeek used vllm 0.11.0, torch 2.8.0, transformers 4.57.1, and bitsandbytes 0.48.1 (NF4 double quantization). On both machines, fresh preflight verified hardware, model artifacts, tokenizers, templates, runtime, and backend configuration identities. Hosted answering was not used, and the models could not call a simulator or any external tool.

Pinned artifact revisions. (1) Qwen/Qwen3-8B. (2) Meta-Llama/Llama-3.1-8B-Instruct. (3) google/gemma-3-27b-it-qat-q4_0-gguf. (4) deepseek-ai/DeepSeek-R1-Distill-Qwen-32B.

I Scoring rules, denominators, and uncertainty

Typed target scoring. Direct, predictive, and counterfactual questions are scored at the target level and then at the whole-question level. A whole question is correct only if every requested target is valid and correct. Units and frames must match. Boolean values require exact Boolean type and value. Scalars use the original absolute-plus-relative tolerance. Vectors use Euclidean distance. Orientation quaternions use the sign-invariant angular distance shown in the main paper.

Explanation factual scoring. The automatic explanation endpoint checks six structured facts: intervention identity, fixed-condition declarations, factual scheduled readout, omission readout, exact-target-equality fact, and existing-scorer distinguishability fact. This score does not determine whether prose is clear, causally careful, or adequate for a human reader.

Clarification scoring. Stage 1 measures whether the model declares a clarification behavior and requests the correct missing slot. Routing is deterministic under the structured rules. Stage 2 is executed only when the first turn supports a recorded reply. Resolved-answer correctness is scored as a typed answer. A model can route successfully and still fail the resolved physical answer.

Prediction assumptions. Every public input states that the simulation is deterministic. It also supplies the model description and its parameters (including contact settings where relevant), the controller law and parameters, initial conditions, the event schedule, and the observed prefix up to the cutoff. Future targets are therefore intended to be inferable by an agent that can integrate these declared dynamics to within the target tolerance, typically 0.02 for velocities and 0.01 for other numerical units. We have not independently established that the public declarations alone reproduce every reference outcome within tolerance, for example by re-simulating from the public projection only. Reaching that tolerance generally requires numerical integration, which a tool-free model must approximate internally. The evaluation therefore does not separate reasoning difficulty from possible information limits or from the lack of computational tools.

Denominators. All selected items stay in primary denominators. Missing, failed, length-limited, unparseable, invalid, and abstained outputs are unsuccessful. Valid-only numerical errors must be paired with coverage; otherwise invalid or missing predictions could disappear from reporting.

Uncertainty. The reported intervals use 10,000 paired whole-trajectory bootstrap replicates with seed 20261009, resampling within domain-family strata. The same sampled groups are used across paired conditions and representations. Intervals are nominal marginal intervals, not simultaneous confidence guarantees. Degenerate bootstrap support is reported as an observed property of the paired sample, not as proof of equivalence.

J Extended results and ablations

This appendix expands the main result tables.

Family	Main required output	Common failure modes
Direct/predictive/ counterfactual	Typed targets with target ID, value, unit, frame	Missing target, wrong JSON type, wrong unit/frame, abstention, invalid JSON, wrong value.
Explanation	Structured facts plus prose fields	Correct readouts but unsupported prose, wrong intervention, missing uncertainty, copied but ungrounded mechanism.
Clarification stage 1	Missing-slot request in schema	Premature answer, wrong slot, invalid schema, disguised answer in prose.
Clarification stage 2	Typed resolved answer after routed reply	Correct route but invalid answer schema, wrong value, wrong unit/frame.

Table 15: Response contracts and representative failure modes.

Condition	Runtime and weights	Reasoning control	Status
Qwen3-8B direct	Transformers, BF16, unquantized	Native thinking disabled	Completed enriched and structured-only.
Qwen3-8B thinking	Transformers, BF16, unquantized	Native thinking enabled	Completed enriched.
Llama-3.1-8B direct	Transformers, BF16, unquantized	Direct prompt	Completed enriched.
Llama-3.1-8B CoT	Transformers, BF16, unquantized	Prompted CoT	Completed enriched.
Qwen3-32B-AWQ off	vLLM, AWQ weights, FP16 runtime	Native thinking disabled	Completed enriched.
Qwen3-32B-AWQ on	vLLM, AWQ weights, FP16 runtime	Native thinking enabled	Completed enriched and structured-only.
Gemma-3-27B QAT direct	llama.cpp, Q4_0 QAT GGUF weights, f16 KV cache	Direct prompt	Completed enriched: 200 initials (181 completed, 19 length-limited).
Gemma-3-27B QAT CoT	llama.cpp, Q4_0 QAT GGUF weights, f16 KV cache	Prompted CoT	Completed enriched: 200 initials (189 completed, 11 length-limited).
DeepSeek-R1-Distill-Qwen-32B	vLLM, NF4 weights, BF16 activations and KV cache	Native reasoning	Completed enriched: 200 initials (200 completed).

Table 16: Model conditions and completion status. Initial-response counts describe generation completion; parseability is reported separately in Table 20.

Reasoning-condition comparisons. Qwen3-8B thinking improves direct, explanation-fact, and clarification endpoints relative to Qwen3-8B direct, while predictive and counterfactual endpoints are lower. Llama CoT improves explanation facts relative to Llama direct but lowers counterfactual success. AWQ-on improves direct and counterfactual point estimates relative to AWQ-off but lowers explanation facts and resolved clarification. Gemma CoT improves predictive (7.5 vs. 2.5) and counterfactual (10.0 vs. 0.0) point estimates relative to Gemma direct, with identical direct, explanation, and clarification endpoints. These are comparisons of full configured conditions.

Parser and contract diagnostics. Llama direct and Llama CoT have 121 and 122 first-stage parser rejections across all families, caused by completed outputs that do not satisfy the frozen final-JSON extraction rule. In the direct condition, 120 of these contain a fenced JSON object surrounded by extra prose. When configurations commit to numerical predictions, most values fall outside tolerance: 35 of 40 for Qwen3-8B direct, 28 of 31 for AWQ-off, and 16 of 17 and 19 of 22 for Gemma direct and CoT. These outputs are not repaired for the primary score. Qwen3-8B thinking and AWQ-on abstain on all predictive items. Thus zero predictive success for those conditions is not the same phenomenon as 40 wrong numerical predictions. Table 5 decomposes predictive and counterfactual outcomes for all ENRICHED conditions.

For the Gemma and DeepSeek conditions, generation completion and parseability are reported separately in Table 20. Gemma's length-limited outputs fall mainly on explanation items (11 per configuration) and on seven Gemma-direct predictive items. On direct readout, all 40 answers of each Gemma configuration are valid JSON but violate the typed answer contract. DeepSeek completes

Condition	Backend	Precision	Temp.	Top-p	Top-k	Max out	Timeout	Thinking
Qwen3-8B direct	Transformers	bf16 / none	0.7	0.80	20	1,024	120 s	off
Qwen3-8B thinking	Transformers	bf16 / none	0.6	0.95	20	8,192	360 s	on
Llama-3.1-8B direct	Transformers	bf16 / none	0.6	0.90	50	1,024	120 s	–
Llama-3.1-8B CoT	Transformers	bf16 / none	0.6	0.90	50	4,096	240 s	–
Qwen3-32B-AWQ off	vLLM	fp16 / AWQ	0.6	0.90	20	1,024	120 s	off
Qwen3-32B-AWQ on	vLLM	fp16 / AWQ	0.6	0.95	20	8,192	360 s	on
Gemma-3-27B QAT direct	llama.cpp	Q4_0 / f16 KV	0.6	0.90	50	1,024	120 s	–
Gemma-3-27B QAT CoT	llama.cpp	Q4_0 / f16 KV	0.6	0.90	50	4,096	240 s	–
DeepSeek-R1-Distill-Qwen-32B	vLLM	bf16 / NF4	0.6	0.95	20	8,192	360 s	–

Table 17: Frozen answering settings. Precision gives the compute precision and the quantization or KV-cache format; Thinking gives the native-thinking switch where one exists. Chat templates are rendered with `add_generation_prompt=True`; Qwen additionally receives the frozen `enable_thinking` flag. No custom stop strings are supplied.

Condition	Input	Direct	Predictive	Counterfact.	Expl. facts	Resolved clar.
Qwen3-8B direct	Enriched	20.0 [10.0,30.0]	12.5 [5.0,22.5]	35.0 [27.5,45.0]	62.5 [52.5,70.0]	10.0 [2.5,20.0]
Qwen3-8B thinking	Enriched	52.5 [40.0,65.0]	0.0 [0.0,0.0]	12.5 [5.0,22.5]	80.0 [70.0,90.0]	25.0 [15.0,35.0]
Llama-3.1-8B direct	Enriched	0.0 [0.0,0.0]	0.0 [0.0,0.0]	17.5 [10.0,25.0]	15.0 [5.0,25.0]	0.0 [0.0,0.0]
Llama-3.1-8B CoT	Enriched	0.0 [0.0,0.0]	0.0 [0.0,0.0]	0.0 [0.0,0.0]	30.0 [17.5,42.5]	0.0 [0.0,0.0]
Qwen3-32B-AWQ off	Enriched	72.5 [67.5,75.0]	7.5 [0.0,15.0]	7.5 [0.0,15.0]	65.0 [57.5,72.5]	32.5 [25.0,40.0]
Qwen3-32B-AWQ on	Enriched	92.5 [85.0,100.0]	0.0 [0.0,0.0]	17.5 [9.9,27.5]	52.5 [37.5,65.0]	30.0 [22.5,40.0]
Gemma-3-27B direct	Enriched	0.0 [0.0,0.0]	2.5 [0.0,7.5]	0.0 [0.0,0.0]	2.5 [0.0,7.5]	0.0 [0.0,0.0]
Gemma-3-27B CoT	Enriched	0.0 [0.0,0.0]	7.5 [0.0,15.0]	10.0 [2.5,17.5]	2.5 [0.0,7.5]	0.0 [0.0,0.0]
DeepSeek-R1-32B	Enriched	60.0 [45.0,75.0]	0.0 [0.0,0.0]	27.5 [17.5,37.5]	27.5 [15.0,40.0]	0.0 [0.0,0.0]
Qwen3-8B direct	Struct.-only, recovered	0.0 [0.0,0.0]	12.5 [5.0,22.5]	35.0 [27.5,45.0]	65.0 [57.5,72.5]	0.0 [0.0,0.0]
Qwen3-32B-AWQ on	Struct.-only	97.5 [92.5,100.0]	2.5 [0.0,7.5]	15.0 [7.5,22.5]	62.5 [50.0,72.5]	32.5 [22.5,42.5]

Table 18: Fixed-denominator endpoints for all conditions on the common 200-item core, with nominal 95% paired whole-trajectory bootstrap intervals (10,000 replicates, stratified by domain and family). Values are percentages over 40 items per family. The Gemma and DeepSeek conditions have no structured-only ablations, so Tables 6 and 19 remain limited to the two ablated conditions.

every initial response, and 29 of its explanation responses are rejected by the frozen final-JSON parser.

Clarification diagnostics. Structured-only Qwen3-8B produces schema-valid first turns on all 40 items: 35 route correctly, and 5 request a wrong supported slot. None of its 35 continuations is correct; 11 violate the resolved schema, and 24 are valid but wrong. Every Qwen and Llama ENRICHED configuration declares a clarification on all 40 items, whereas Gemma direct, Gemma CoT, and DeepSeek do so on 1, 2, and 3 items (2.5–7.5%). Gemma direct routes no item. Gemma CoT routes one: its structured selector correctly requests the execution condition while its readable question asks for a unit, and automatic routing follows the structured selector. Its continuation answers a 1-s query with the observed 0.2-s angle, which is incorrect. DeepSeek routes one item, but its attempted continuation saved no answer.

Restart recovery. The Qwen3-8B structured-only run originally lost 35 initial outputs to an external restart. After an authorized recovery with identical requests and settings, its apparent explanation advantage for enriched input disappeared. All reported values for this condition are the recovered values.

K Resource-evaluation evidence and human review

The resource evaluation asks whether SimPhysQA provides the components needed to build and evaluate AI systems over the five families.

Prepared cohorts. The original admitted predictive source contains 11,105 QA records across 6,340 trajectory groups. The prepared enriched union is bounded: 56 development items, 140 annotation-audit items, and a 400-item held-out pool that contains the nested 200-item core. The 200-item core is the evaluated subset for every reported model result; the remaining pool items are part of the dataset inventory. The prepared

Comparison	Family	Difference and interval	Interpretation
Qwen3-8B structured-only minus enriched	Direct	-20.0 [-30.0, -10.0]	Enriched input helps direct readout for this condition.
Qwen3-8B structured-only minus enriched	Predictive	0.0 [0.0, 0.0]	No observed difference.
Qwen3-8B structured-only minus enriched	Counterfactual	0.0 [0.0, 0.0]	Recovered data eliminate the original apparent enriched advantage.
Qwen3-8B structured-only minus enriched	Explanation facts	+2.5 [0.0, +7.5]	Small structured-only point advantage after recovery.
Qwen3-8B structured-only minus enriched	Clarification	-10.0 [-20.0, -2.5]	Enriched input helps resolved clarification for this condition.
AWQ-on structured-only minus enriched	Direct	+5.0 [-2.5, +12.5]	No clear enriched advantage.
AWQ-on structured-only minus enriched	Predictive	+2.5 [0.0, +7.5]	Small point difference, interval touches zero.
AWQ-on structured-only minus enriched	Counterfactual	-2.5 [-15.0, +10.0]	No clear difference.
AWQ-on structured-only minus enriched	Explanation facts	+10.0 [-7.5, +27.5]	No clear difference.
AWQ-on structured-only minus enriched	Clarification	+2.5 [-5.0, +12.5]	No clear difference.

Table 19: Matched representation ablations. Negative values favor enriched input. The ablations are available only for Qwen3-8B direct and Qwen3-32B-AWQ-on.

Condition	Completed	Length-limited	Completed & parseable
Gemma-3-27B direct	181/200	19/200	139/200
Gemma-3-27B CoT	189/200	11/200	151/200
DeepSeek-R1-32B	200/200	0/200	134/200

Table 20: Initial-response execution status for the Gemma and DeepSeek conditions. All selected items stay in the primary denominators.

union contains 596 QA items across 556 trajectory groups. These counts are included for provenance but should not displace the main contribution narrative.

Family coverage. The prepared union contains 124 direct, 124 predictive, and 116 each of counterfactual, explanation, and clarification records. The nested main core contains 40 items per family, 50 per domain, and 10 items per domain-family cell.

Annotation audit: design. Three reviewers independently assessed the same 140 annotation packets: four domains × five families × seven packets, for 420 individual ratings. There was no separate calibration set, and original ratings were submitted before any discussion. Each packet receives one primary judgment, *acceptable* or *material_error*, within a focused scope of five criteria: physical/global context, trajectory language, question wording and type, timestamp language, and state-event descriptions. A material error is one that could change the physical interpretation, the target entity, time, unit, or frame, or the intended question, or one that asserts an unsupported fact or effect. Minor awkward wording is recorded as a comment and does not make a packet unacceptable. For each material error, reviewers record the criterion, an error category, a source location, and a comment. The complete frozen reviewer guide is reproduced below.

Annotation audit: results. Table 21 reports the share of the 140 packets that each reviewer judged acceptable. The mean of the three individual rates is approximately 90%. These are rates of individual ratings. They are neither the share of packets accepted unanimously nor a measure of inter-rater agreement, which we do not report here.

Reviewer	Acceptable	Material error
R1	100%	0%
R2	83%	17%
R3	87%	13%
Mean of individual rates	≈90%	≈10%

Table 21: Focused annotation audit: per-reviewer share of the same 140 packets judged acceptable. These are rates of individual ratings, not unanimous packet acceptance or inter-rater agreement.

Annotation audit: scope. The audit assesses the model-facing annotation layer. Gold answers, explanations, and private traces may be consulted to corroborate this scope, but the audit does not establish the correctness of every reference answer, verification trace, or model explanation, and it does not rate answering-model performance. With seven packets per domain–family cell, it does not support per-cell estimates. Discussion, adjudication, and any proposed corrections are kept separate from the frozen packets and original ratings.

```
# Focused reviewer guide /1.0.0

Review independently using your assigned R1, R2 or
     R3 slot. All three reviewers
inspect the same140 items; slots do not specify
     personal identities. No separate
calibration set. Copy your frozen CSV form before
     editing. Submit original ratings
before discussion. Blank judgments are uncollected,
     never acceptable ratings.

Primary judgment: acceptable or material_error **
     within the focused scope**:
physical/global context, trajectory language,
     question wording/type, generated
timestamp language and state-event descriptions. A
     material error could change
the physical interpretation, target/entity/time/
     unit/frame or intended question,
or asserts an unsupported fact/effect. Optional
     gold answers, explanations, traces
and extra private verification corroborate this
     scope; they are not automatically
part of a comprehensive answer-quality audit. Do
     not rate answering-model performance.

Supply each criterion as acceptable/material_error
     /not_applicable:
global_context; trajectory_language;
     question_type_wording; timestamp_language;
state_event_descriptions. Use not_applicable only
     for a criterion that has no
applicable content (e.g. no declared events);
     missing required content is an error.
Global context describes physical setup;
     trajectory text describes the observed
window. They need not be duplicate paraphrases.
     Timestamp text is generated
language joined by state_id, distinct from the
     generic original state.context.

Error categories: factual_contradiction,
     unsupported_claim, wrong_entity,
wrong_time, wrong_unit, wrong_frame,
     misleading_event_semantics,
question_type_mismatch, readability,
     missing_required_field, other. Minor awkward
wording may be recorded in comments while the item
     remains acceptable. Use the
readability error category for material ambiguity/
     illegibility. For material_error
record at least one material criterion, category,
     source location (JSON pointer,
state/event/link ID or question field) and an
     explanatory comment. Separate
multiple categories/locations with semicolons.
     Criterion CSV syntax:
`global_context=acceptable;trajectory_language=
     acceptable;question_type_wording=acceptable;
     timestamp_language=acceptable;
     state_event_descriptions=not_applicable`.

Assess statements against original measurements/
     relations, entities and declarations,
with their units/frames and exact temporal
     relationships. Scheduled future events
are declarations, not evidence they were applied
     or caused an effect. Ended timing
does not mean effects ended; torque/force limits
     do not establish realized values.
Shared event text repeats per state with that
     state's actual relationship.
Explicitly labeled unresolved semantics or
     limitations are acceptable if the prose
faithfully conveys what is established and
     uncertain. Asserted resolution without
evidence, concealed uncertainty that changes
     interpretation, or contradicted source
is material_error. Never convert a declared
     limitation into an invented fact.

Direct asks an observed readout. Predictive asks
     future outcomes. Counterfactual
asks the verified omission world; a Boolean can
     remain unchanged and a numerical
contrast can be within tolerance. Explanation
     intentionally supplies its selected
comparison outcomes in a separate task field, not
     in observed states; assess
question/family framing and public descriptions,
     not automatically every private
gold claim. Clarification is an explicit absent
     query selector with two compatible
incompatible-answer completions. Inspect labeled
     initial question/view, authored
clarification, recorded author reply and resolved
     question/view. They are reference
turns, not an executed model conversation. The
     reply supplies information, not gold.

Freeze original submissions immutably. Discussion,
     adjudication, proposed annotation
corrections and any later rubric revision live
     separately and do not change the
packets or original independent ratings. Do not
     begin formal review from the
development extraction-check index or header-only
     preparation templates.
```

Model-answer prose study. The prospective model-answer prose study freezes eight explanation tasks, two per domain, for all nine enriched conditions, giving 72 planned responses. All 72 now exist: 48 from the six Qwen and Llama conditions and 24 from the Gemma and DeepSeek conditions. Six of the latter are length-limited (two from Gemma direct, four from Gemma CoT) and remain in the subset unchanged. Human assessment of model explanation prose is a separate study whose two independent blinded assessor forms remain uncollected, so the paper reports no human prose-adequacy result.

L Responsible NLP checklist mapping

This section maps the submission to common Responsible NLP review concerns.

Data and documentation. The manuscript identifies the physical domains, QA families, public/private split, validation checks, release scope, and missing evidence boundaries. It distinguishes the original predictive inventory from the prepared five-family enriched union.

Annotation. The paper describes automatic enrichment, reproduces every executed generation, repair, and evaluation prompt, and describes the

focused annotation audit protocol. It reports per-reviewer acceptance rates from the focused annotation audit without presenting them as inter-rater agreement, and it does not claim human prose-quality judgments.

Models. The paper lists all nine evaluated conditions (five models), runtime classes, reasoning controls, generation settings, execution status, and pinned artifact revisions.

Evaluation. The paper reports fixed denominators, missingness handling, parsing diagnostics, typed scoring, clarification routing, explanation factual scoring, and bootstrap intervals. It explicitly avoids pooling incompatible endpoints into one headline score.

Compute. The paper reports serial local execution on two workstations, their verified runtimes, and frozen generation settings. It does not claim measured GPU utilization.

Risks and intended use. The resource is intended for research on grounded scientific QA and evaluation methodology. It should not be used as evidence that a model is safe for real-world robotic, vehicle, or physical control deployment.

Open limitations. The main open evidence gaps are independent model-answer prose review, exact per-reviewer counts and agreement statistics for the annotation audit, full-release enrichment, and broader physical-domain coverage.

M Readable examples for all five families

This section gives schematic examples of how the same kind of trajectory evidence becomes different QA tasks. The examples are deliberately written in ordinary language; exact item text in the dataset is preserved in the frozen request files.

M.1 Direct readout example

Public evidence. The public record shows a tethered-load trajectory up to time $t = 0.4$ s. One public state contains the load swing angle with entity `load`, quantity `swing_angle`, unit radians, frame `world`, and the exact timestamp.

Question. What is the load swing angle at $t = 0.4$ s?

Expected behavior. The model should locate the public state, return the target ID and a numeric value with the declared unit and frame, and not invent a different time or object.

Scoring. The answer is correct only if the JSON target is valid, the unit/frame match, and the scalar value is within the target tolerance.

M.2 Predictive example

Public evidence. The model sees vehicle states up to the observation boundary and the declared braking controller. The future terminal state is not public.

Question. What will the car's forward velocity be at $t = 2.0$ s?

Expected behavior. The model must answer from the visible prefix and declared controls. It cannot read the future value from the prompt because that value is private.

Scoring. The prediction is compared against the private admitted reference outcome. Abstention is counted as unsuccessful in the primary endpoint.

M.3 Counterfactual example

Public evidence. The model sees the factual scenario and a task statement that disables a scheduled intervention, such as omitting a push or braking command. It does not see the alternative rollout.

Question. If the scheduled push were omitted, would block 2 remain supported at the target time?

Expected behavior. The model answers for the specified alternative condition, not the original factual trajectory. The answer may be unchanged relative to the factual world; unchanged outcomes remain legitimate.

Scoring. The answer is compared against the verified matched alternative. The factual replay and alternative admission evidence remain private.

M.4 Explanation example

Public evidence. The model is intentionally shown two selected readouts: the factual scheduled outcome and the matched omission outcome. The prompt states which intervention changes and which conditions remain fixed.

Question. Explain the comparison between the factual and omission outcomes.

Expected behavior. The model should identify the intervention, fixed conditions, two readouts, whether they are exactly equal, whether the existing scorer distinguishes them, and a careful interpretation with uncertainty.

Scoring. The automatic endpoint checks structured facts only. A separate human study is required for prose adequacy.

M.5 Clarification example

Public evidence. The initial public view leaves a selector unresolved. For example, a question may not specify which object, time, or execution condition is intended.

Question. Did it succeed?

Expected behavior. The model should not guess. It should ask for the missing selector. After receiving the recorded reply, it should answer the resolved task.

Scoring. Stage 1 is judged by structured gap recognition and slot match. Stage 2 is judged by typed answer correctness. The stages are reported separately.

N Prompt and serialization discipline

Why exact serialization matters. The same task can become easier, harder, or ambiguous if fields are reordered, omitted, paraphrased, or accidentally merged. SimPhysQA therefore preserves exact rendered requests, model chat templates, tokenization checks, raw responses, and parser/scorer versions. The paper's tables summarize this evidence; the execution package stores the exact request bytes.

Public request assembly. Each public request has a condition identifier, a sample identifier, no tools, and a message containing instructions plus the canonical public task view. Direct/predictive/counterfactual requests append the typed-target response contract. Explanation requests use the explanation-specific response contract. Clarification requests use the initial ambiguous public view and a schema for requesting missing information.

Structured-only projection. The structured-only ablation removes derived trajectory, timestamp, and event descriptions. It preserves the underlying task facts, original context, controls, question, answer specification, supplied explanation comparisons, and clarification ambiguity. It is therefore an input-representation ablation, not a removal of answerability.

No answer repair. The primary evaluation does not repair outputs, substitute gold turns, reopen failed answers, or score alternate parser interpretations. Diagnostic reports may classify rejected output surfaces, but those classifications do not change the primary score.

O Additional interpretation notes for reviewers

A high direct score is not physical understanding. Direct QA often tests whether the model can locate visible evidence and return it in the required contract. This is useful, but it does not imply future prediction or counterfactual reasoning.

A high explanation-fact score is not prose adequacy. The automatic explanation endpoint is intentionally narrow. It checks whether the model reports the supplied comparison facts faithfully. It does not judge whether the model gives a convincing, causally careful, or pedagogically useful explanation.

Clarification has two failures. A model can fail by answering too early, asking for the wrong slot, or failing to answer after receiving the correct selector. SimPhysQA reports these pathways separately because they correspond to different assistant behaviors.

Low predictive success is not a general inability. Predictive and counterfactual scores measure end-to-end success under the supplied public evidence and a tool-free interface. Abstentions, output-contract failures, and incorrect values are reported separately (Table 5). Because public-input sufficiency has not been independently established and no computational tools are available, low scores do not separate reasoning difficulty from information limits or the absence of numerical integration.

Ablation scope is limited. Only two structured-only ablations are completed. This is enough to test whether language enrichment helps those conditions, but not enough to claim a universal input-representation effect. The paper therefore reports selective evidence rather than a broad enrichment-benefit claim.

Artifact	Preserved information	Reason
Prepared request	Exact public prompt, condition ID, input representation, response contract	Ensures the benchmark can be rerun and audited.
Token/render check	Model-specific template and token counts	Detects context overflow or serialization differences.
Raw output	Complete returned text and termination evidence	Prevents silent cleanup or selective parsing.
Parsed answer	Final JSON object or parser failure	Separates surface format from scientific correctness.
Scoring row	Typed validity, target correctness, errors, status	Makes denominator and failure policy inspectable.
Timing/memory receipt	Loading, generation, process lifecycle, memory samples where available	Documents execution feasibility without overclaiming GPU utilization.

Table 22: Execution artifacts that support reproducibility and error analysis.

Mixed results are expected for a useful resource. A resource that exposes different capabilities should not necessarily produce a clean monotonic leaderboard. The value of SimPhysQA is that it can show when reading, predicting, explaining, and clarifying diverge.

P What the release supports and does not support

Supported. The current evidence supports the claim that SimPhysQA provides a bounded, five-family, simulator-grounded QA resource with enriched trajectory records, public/private separation, family-specific evidence contracts, and local-model results for nine enriched conditions of five models and two structured-only ablations on a common 200-item core.

Not supported yet. The current evidence does not support claiming full enrichment of every original predictive record, human explanation-quality conclusions, semantic entailment of every generated sentence, sufficiency of the public inputs to determine every future target, real-world physical reliability, broad out-of-distribution dialogue competence, or a universal language-enrichment benefit.

Why this honesty strengthens the paper. A resource paper is judged not only by positive results but by whether its evidence boundaries are clear enough for other researchers to trust and reuse. The paper therefore treats unavailable or pending evidence as part of the scientific record rather than smoothing it away.